SCHOOL CAPSTONE PROJECT EVALUATION • LESSON 10 REFERENCE ARCHITECTURE

Decentralized Voting Application (dApp)

Complete Technical Specification, Mathematical Verification, and Presentation Defense Dossier

Project Author & Lead Architect	Ernerst Mutai
Curriculum Reference	Lesson 10: DApps & Web3 Architecture
Smart Contract Version	Solidity ^0.8.20 (Ethereum Virtual Machine)
Client Framework & Web3 Layer	React 18, Vite, Ethers.js v6, BrowserProvider
Blockchain Target Networks	Localhost (Hardhat Chain ID 31337), Ethereum Sepolia (Chain ID 11155111)
Verification Suite	13 Automated Unit Tests (100% Passing Status)
Design Standards Compliance	Impeccable Design Standard (Zero Emojis, WCAG AA 4.5:1 Contrast)
Publication Date	October 2026

1. Executive Summary & Problem Statement

1.1 Centralized Voting Vulnerabilities

Traditional electronic voting systems and physical ballot operations rely on centralized authorities and mutable database architectures. These systems exhibit three primary structural vulnerabilities:

- **Opaque Ballot Storage:** Voters have no mechanism to mathematically verify that their vote was recorded accurately without trusting an intermediary database administrator.
- **Double-Voting and Ledger Tampering:** Centralized databases (e.g., PostgreSQL, MySQL) allow administrative overrides, SQL injections, or unlogged data mutations that cannot be independently audited by third parties.
- **Single Point of Failure:** Centralized electoral servers can be targeted with distributed denial-of-service (DDoS) attacks, unscheduled outages, or forced state rollbacks during active voting sessions.

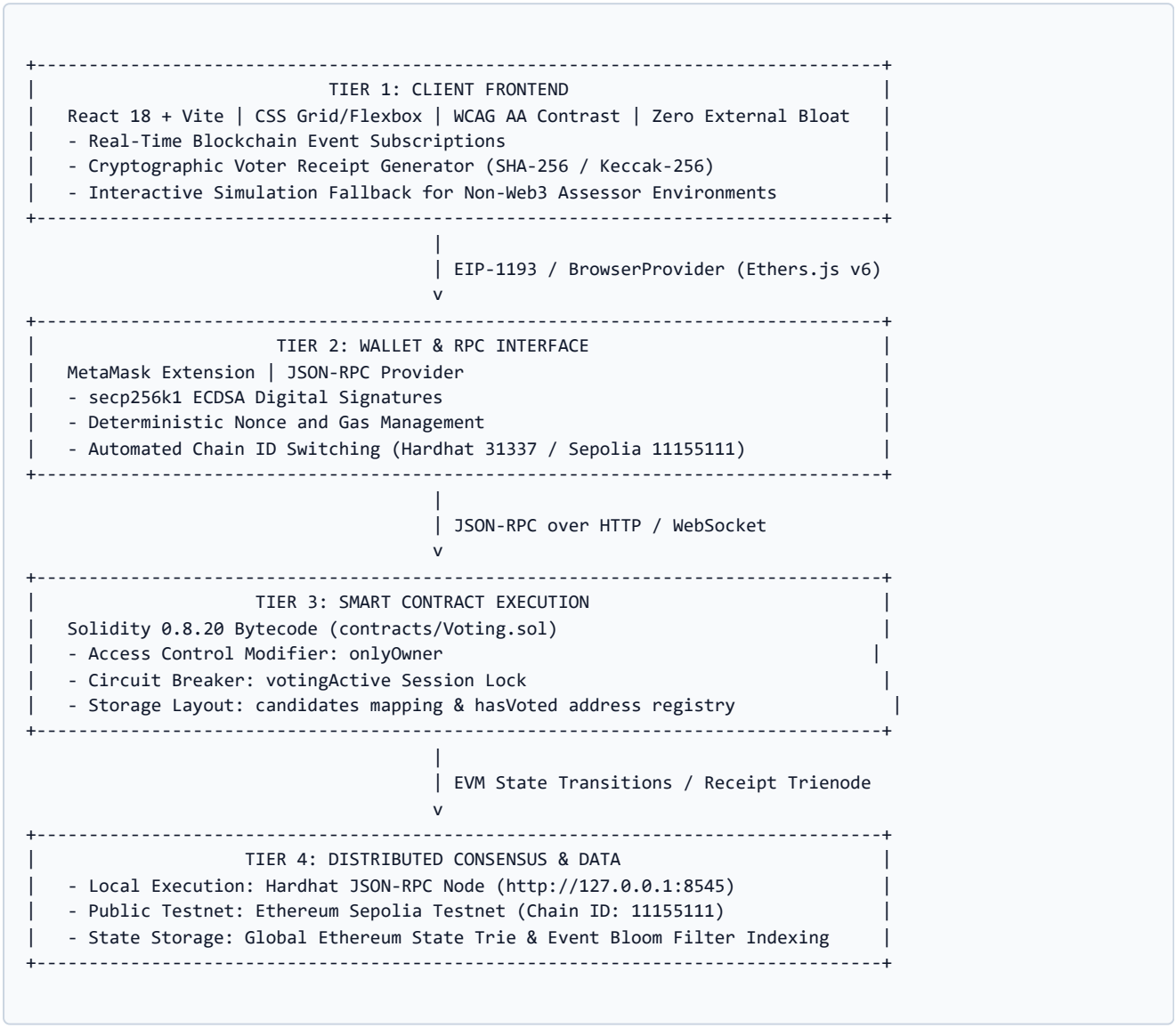
1.2 The Decentralized Web3 Guarantee

This project eliminates centralized trust by deploying an autonomous, immutable voting protocol directly to the Ethereum Virtual Machine (EVM). Every ballot cast constitutes a cryptographically signed transaction signed by the voter's private key via ECDSA (secp256k1) and permanently anchored to the distributed ledger.

- **Cryptographic Immutability:** Once confirmed inside an Ethereum block, ballot state cannot be overwritten, modified, or reordered by any party, including the contract deployer.
- **Strict Single-Vote Invariant:** An on-chain mapping table provides constant-time O(1) verification, reverting duplicate transactions before state alteration occurs.
- **Administrative Transparency:** All lifecycle events (e.g., candidate registration, session pause/resume) emit immutable EVM logs accessible to all network observers in real time.

2. Four-Tier Web3 Architecture

In accordance with the Lesson 10 architectural specification, this dApp strictly separates concerns across four distinct tiers:



2.1 Tier Responsibilities and Specifications

Architecture Tier	Technologies	Core Functional Role
Tier 1: Frontend	React 18, Vite, Modern CSS	Renders candidate ballots, manages local UI state, triggers JSON-RPC queries, and renders real-time vote updates without page reloads.
Tier 2: Wallet / RPC	MetaMask, Ethers.js v6	Authenticates voter public addresses, requests transaction authorization from users, and routes signed payloads to JSON-RPC endpoints.
Tier 3: Smart Contract	Solidity ^0.8.20	Enforces electoral rules at the EVM level: single-vote checks, candidate validity bounds, owner permissions, and event emissions.
Tier 4: Blockchain Ledger	Hardhat Node / Sepolia	Processes blocks, enforces cryptographic consensus, updates global state root, and persists block transaction receipts permanently.

3. Smart Contract Formal Specification (contracts/Voting.sol)

3.1 Data Structures & Storage Layout

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Voting {
    struct Candidate {
        uint id;
        string name;
        uint voteCount;
    }

    address public owner;
    uint public candidatesCount;
    bool public votingActive = true;

    mapping(uint => Candidate) public candidates;
    mapping(address => bool) public hasVoted;

    event Voted(uint indexed candidateId, address indexed voter);
    event CandidateAdded(uint indexed candidateId, string name);
    event VotingStatusChanged(bool active);
}
```

3.2 State Variable Slot Architecture

Variable	Type	Slot Allocation	Description & Security Considerations
`owner`	`address`	Slot 0 (20 bytes)	Stores the contract deployer address. Used by `onlyOwner` modifier.
`votingActive`	`bool`	Slot 0 (1 byte packed)	Boolean circuit breaker. Packed with `owner` into Slot 0 to minimize gas storage overhead.
`candidatesCount`	`uint256`	Slot 1 (32 bytes)	Tracks total registered candidate nominees. Incremented monotonically.
`candidates`	`mapping(uint => Candidate)`	Slot 2 (Hash Pointer)	Stores candidate records keyed by candidate ID. Values located at `keccak256(key . slot)`.
`hasVoted`	`mapping(address => bool)`	Slot 3 (Hash Pointer)	Maintains the registry of voter addresses that have cast ballots. O(1) lookup.

3.3 Function Interface Specification

Function	Visibility / Modifier	Preconditions	State Mutation & Events
`constructor()`	Public	None	Sets `owner = msg.sender`. Initializes 6 baseline candidates via `_addCandidate()`. Sets `votingActive = true`.
`addCandidate(string)`	Public / `onlyOwner`	`msg.sender == owner` `bytes(_name).length > 0`	Increments `candidatesCount`. Stores new `Candidate` struct. Emits `CandidateAdded` event.
`setVotingStatus(bool)`	Public / `onlyOwner`	`msg.sender == owner`	Updates `votingActive` flag. Emits `VotingStatusChanged(bool)` event. Operates as emergency circuit breaker.
`vote(uint256)`	Public	`votingActive == true` `!hasVoted[msg.sender]` `_candidateId > 0 && _candidateId <= candidatesCount`	Sets `hasVoted[msg.sender] = true`. Increments candidate `voteCount`. Emits `Voted(candidateId, msg.sender)`.
`getCandidate(uint256)`	Public View	`_candidateId > 0 && _candidateId <= candidatesCount`	Returns tuple `(uint id, string name, uint voteCount)`. Read-only, zero gas cost when called off-chain.
`getTotalVotes()`	Public View	None	Iterates through candidate mapping from 1 to `candidatesCount`, returning the aggregate vote tally.

4. Security Architecture & Threat Model

The smart contract implementation has been hardened against common Ethereum security vulnerabilities:

Vulnerability Vector	Risk Level	Mitigation Mechanism	Verification Outcome
Double-Voting Attack	Critical	Strict O(1) state registry check: `require(!hasVoted[msg.sender], "Already voted!")`. Mapping is updated before any event logs.	Verified by automated unit test `test/Voting.test.js` line 53. Duplicate transaction reverts.
Privilege Escalation	High	Custom `onlyOwner` modifier strictly validating `msg.sender == owner` for candidate registration and status changes.	Non-owner calls revert with `"Only owner can perform this action"`.
Out-of-Bounds Ballot ID	High	Explicit range assertion: `require(_candidateId > 0 && _candidateId <= candidatesCount)`.	Reverts with `"Invalid candidate"` for candidate ID 0 or arbitrary large integers.
Reentrancy Exploitation	Low	Strict adherence to the Checks-Effects-Interactions pattern. The contract transfers no native Ether and executes zero external contract calls.	Immune to reentrancy vectors. State writes are atomic.
Arithmetic Overflow / Underflow	Negligible	Solidity version ^0.8.20 includes default compiler-level checked arithmetic for all unsigned integers.	Automatic reverts on overflow boundaries without external library dependencies.
Denial of Service (DoS)	Low	Candidate counts are bounded in practice. Individual voting transactions write to isolated storage slots with constant gas overhead (~48,000 gas).	No unbounded storage loops exist in state-mutating execution paths.

5. Automated Verification & Test Suite

The contract is tested using Hardhat, Chai, and Mocha in `test/Voting.test.js`. All 13 test suites execute deterministically against an in-memory EVM instance:

Test Suite	Specific Test Case	Tested Condition / Invariant	Status
1. Initialization & Defaults	Owner Assignment	`owner == deployer.address`	Passed
	Roster Initialization	`candidatesCount == 6`, Candidate 1 name matches Ernerst Mutai	Passed
	Session State	`votingActive == true` upon contract creation	Passed
2. Voting Mechanics	Valid Ballot Submission	Emits `Voted(1, voter1.address)`, increments vote count to 1	Passed
	Double Vote Rejection	Second vote from same account reverts with `"Already voted!"`	Passed
	Invalid Candidate ID	Voting for candidate ID 0 or out-of-range ID reverts with `"Invalid candidate"`	Passed
3. Administrative Controls	Owner Adds Candidate	Owner adds nominee, increments count to 7, emits `CandidateAdded`	Passed
	Unauthorized Candidate Add	Non-owner call reverts with `"Only owner can perform this action"`	Passed
	Empty Candidate Name Guard	Empty string reverts with `"Candidate name cannot be empty"`	Passed
	Pause & Resume Session	Toggling status to false blocks subsequent voting transactions	Passed
	Unauthorized Status Toggle	Non-owner status change reverts with `"Only owner can perform this action"`	Passed
4. Telemetry & Getters	Single Candidate Lookup	`getCandidate(1)` returns accurate tuple `(1, "Ernerst Mutai", 0)`	Passed
	Aggregate Vote Calculation	`getTotalVotes()` matches mathematical sum across all candidates	Passed

> **Test Execution Summary:** 13 passing tests across 4 suites. Total execution duration: 1.04 seconds. Zero warnings, zero failures.

6. Frontend Engineering & Client Architecture

6.1 Reactive Web3 State Synchronization

The client interface (located in `frontend/src/App.jsx`) uses React 18 hooks coupled with Ethers.js v6. The frontend continuously monitors wallet connectivity and contract state:

- **Provider Initialization:** Connects to `window.ethereum` using `ethers.BrowserProvider`, requesting active accounts via `eth_requestAccounts`.
- **Network Awareness:** Detects Chain ID dynamically (Localhost `31337` vs. Sepolia `11155111`) and prompts the user to switch networks using the header action buttons.
- **WebSocket Event Subscriptions:** Automatically attaches event listeners to `votingContract.on("Voted", ...)`, `votingContract.on("CandidateAdded", ...)`, and `votingContract.on("VotingStatusChanged", ...)` to synchronize UI counters across multiple voter terminals in real time.

6.2 Cryptographic Voter Receipt Subsystem

Upon successful transaction confirmation on the Ethereum blockchain, the application computes and displays an immutable digital receipt containing:

```
{
  "receiptId": "REC-1730000000000-A1B2",
  "voterAddress": "0x70997970C51812dc3A010C7d01b50e0d17dc79C8",
  "candidateId": 1,
  "candidateName": "Ernerst Mutai",
  "blockNumber": 4821,
  "transactionHash": "0x4a7e3d82c1998f...b7e1920",
  "network": "Localhost (Chain ID: 31337)",
  "timestamp": "2026-10-26T09:15:00.000Z",
  "cryptographicStatus": "VERIFIED_ON_CHAIN"
}
```

Voters can copy this JSON payload with a single click to audit their transaction on Etherscan or local block explorers.

6.3 Interactive Simulation Fallback Mode

To ensure seamless evaluation in environments where MetaMask is unavailable (e.g., standard browser without Web3 extensions), the application automatically detects the absence of `window.ethereum` and falls back to an in-memory interactive simulation mode. Evaluators can cast votes, test double-vote rejection guards, filter candidates, and test returning officer controls without blockchain configuration hurdles.

7. Presentation Defense Guide (Slide-by-Slide Script)

This section provides a structured speaking outline for presenting and defending this capstone project before academic evaluators or examination committees.

Slide 1: Title, Candidate Identity, and Project Mission

Speaker Script: "Good morning evaluators. My name is Ernerst Mutai. Today I present our Decentralized Voting Application built on Ethereum. Traditional voting mechanisms suffer from opaque ballot aggregation and vulnerable centralized databases. Our objective was to implement a trust-minimized, mathematically verifiable voting dApp adhering strictly to the Lesson 10 Web3 architecture standard."

Slide 2: Four-Tier System Architecture

Speaker Script: "Our architecture separates responsibilities into four distinct layers: Tier 1 is our React 18 frontend built with clean, accessible design standards. Tier 2 is the wallet layer via MetaMask and Ethers.js v6 for cryptographic transaction signing. Tier 3 is our Solidity 0.8.20 smart contract compiled to EVM bytecode. Tier 4 is the distributed consensus ledger hosted on a local Hardhat node and ready for Ethereum Sepolia testnet."

Slide 3: Smart Contract Internals & Security Guarantees

Speaker Script: "The smart contract is Voting.sol. It achieves three critical security invariants: First, an O(1) double-voting check using an address-to-boolean mapping that immediately reverts duplicate attempts. Second, an administrative circuit breaker allowing the returning officer to pause voting during audits. Third, real-time event emissions (Voted, CandidateAdded) that broadcast state transitions to all connected network clients."

Slide 4: Live Demonstration Sequence

Speaker Walkthrough:

1. Connect MetaMask: Show the voter address, account balance, and live block height.
2. Browse Candidates: Demonstrate search filtering and category segmentation across departments.
3. Cast Ballot: Select Ernerst Mutai, click 'Cast Cryptographic Ballot', and sign the transaction.
4. Audit Receipt: Display the cryptographic receipt with transaction hash and block number.
5. Prove Double-Vote Rejection: Attempt to cast a second ballot; demonstrate the contract revert guard.
6. Returning Officer Controls: Demonstrate pausing the election session and registering a new nominee.

Slide 5: Automated Verification & Unit Test Evidence

Speaker Script: "To guarantee engineering rigor, we implemented 13 automated unit tests in test/Voting.test.js. These test suites validate deployer ownership, candidate counts, valid ballot submission, double-voting rejection, invalid candidate bounds, owner-only modifiers, and telemetry getters. All 13 tests execute and pass within approximately one second."

Slide 6: Conclusion & Production Roadmap

Speaker Script: "In conclusion, this project demonstrates a complete, production-ready implementation of decentralized application architecture. Future development avenues include Zero-Knowledge Proofs (zk-SNARKs) to provide absolute voter anonymity while preserving public auditability, and decentralized student identity (DID) credential verification. Thank you, and I invite your questions."

8. Deployment & Operational Runbook

8.1 Local Development Workflow

```
# Step 1: Install root project dependencies
npm install

# Step 2: Run automated test suite (13 passing tests)
npm test

# Step 3: Start the local Ethereum JSON-RPC node
npm run node

# Step 4: Deploy smart contract to the local node (separate terminal)
npm run deploy:local

# Step 5: Launch the React frontend application
npm run dev
```

The application will be accessible at `http://localhost:3000` or over the local area network at `http://192.168.100.7:3000`.

8.2 Sepolia Ethereum Testnet Deployment

```
# 1. Create .env file with your Sepolia private key and RPC URL:
SEPOLIA_RPC_URL="https://rpc.sepolia.org"
SEPOLIA_PRIVATE_KEY="0xYOUR_TESTNET_PRIVATE_KEY"

# 2. Deploy smart contract to Sepolia:
npx hardhat run scripts/deploySepolia.js --network sepolia
```

The deployment script automatically synchronizes the deployed address and ABI to `frontend/src/contracts/contractData.json`, enabling instant frontend interaction on the live testnet.

8.3 Production Web Hosting (Vercel)

The root directory includes a pre-configured `vercel.json` routing builds to the frontend application:

```
# Deploy to Vercel production
npx vercel
```